

第四章 面向对象需求分析方法

① 本章速览

核心问题

- 如何从用户视角把"模糊需求"转为"系统要做什么"的可视化模型?
- OOA 阶段产出哪两个模型? 各自描述什么?
- 用例模型由哪 4 个部分组成? 哪些必选、哪些可选?
- 领域模型中的"概念类"与软件类的区别?
- 用例之间的 包含 <> 与 扩展 <> 怎么区分?
- 类之间 6 种关系 (依赖/关联/聚合/组合/继承/实现) 的画法与强弱?
- 操作契约的"后置条件三要素"是什么?

核心结论

- OOA 产出: **领域模型 + 用例模型**; OOD 产出: 设计模型。
- 用例模型 = 用例图 + 用例说明 + 系统顺序图(SSD) + 操作契约 (后两个是**附加**, 简单用例可省)。
- 领域模型用 **UML 类图**表示概念类, 业务流程用 **活动图**表示。
- SSD 只画两类对象: **Actor + 系统黑盒**, 不画控制器/DAO/DB。
- 操作契约后置条件三要素: **对象创建/消除、关联形成/断开、属性值修改**。
- include 是"必发"步骤抽取; extend 是"条件触发"分支抽取。

高频考点 (5 年题库统计)

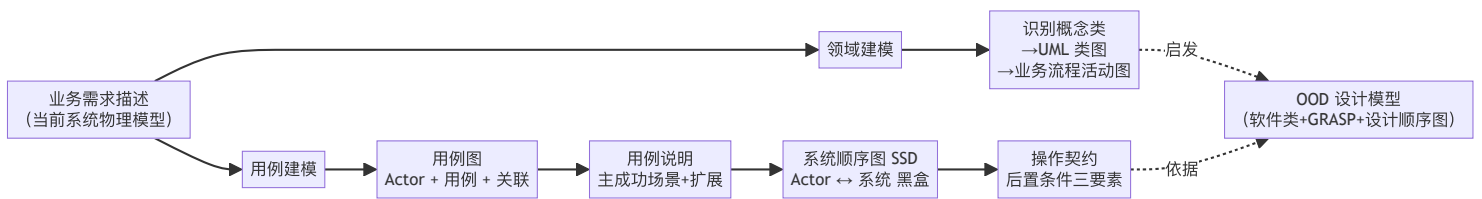
判断 单选 简答 综合大题

1. **用例模型 4 要素** (必考填空/单选/简答, 2023-2024、2024-2025)
2. **操作契约后置条件三要素** (必考单选/简答, 2013-2014、2018-2019、2021-2022、2023-2024)
3. **包含 vs 扩展关系** (必考单选/判断, 2010-2011、2013-2014、2024-2025)
4. **领域模型 vs 软件对象** (必考判断/简答, 2007-2008、2015-2016、2021-2022)
5. **SSD vs 设计顺序图** (必考判断/单选, 2009-2010、2010-2011、2011-2012)
6. **综合大题**: 医院挂号/ATM/电子商务/订票/食堂一卡通/写诗机器人 (2007-2025 全覆盖)

② 题目关键词导航

题干关键词	考点定位	教材锚点	典型题型
"用例模型由几个部分构成"	用例模型 4 要素	§4.5.2 用例模型的组成结构	填空 / 单选
"操作契约后置条件包含哪些"	后置条件三要素	§4.5 操作契约表 4-8	单选 / 简答
"包含关系 vs 扩展关系"	用例间关系判断	§4.5.4 用例之间的关系	单选 / 判断
"以下哪项不能作为角色"	Actor 定义	§4.5.3 角色 Actor	单选
"领域模型属于软件模型吗"	领域模型本质	§4.4.3 与软件对象的区别	判断
"系统顺序图属于哪个模型"	SSD 归属 (用例模型)	§4.5.2 + §4.5.5 SSD	单选 / 判断
"OOA 不包括哪项工作"	OOA vs OOD 范围	§4.1.1 OOA 五步	单选
"画用例图/领域模型/SSD/操作契约"	综合大题全要素	§4.4~§4.5 全章	综合大题
"业务流程用什么图"	活动图≠用例图	§4.4.2 表示	判断
"UML 4+1 视图"	4+1 视图作用	§4.2.2 UML 概述	简答 / 单选

③ 知识主线



说明: 领域模型 ("有什么") + 用例模型 ("做什么") = OOA 双视图, 二者相辅相成、相互校验。设计模型在第 7 章。

④ 核心知识点详解 (PPT 主导)

4.1 UML 概述

UML 是什么

- **统一建模语言** (Unified Modeling Language), 由 Booch、Jacobson、Rumbaugh 三人发起, 在 Booch、OMT、OOSE 方法基础上集众家之长形成, 由 OMG 采纳为**业界标准**。
- UML 是**图形化建模语言**, 是面向对象分析与设计的**标准表示**。
- **不是程序设计语言**、**不是工具规格说明**、**不是过程或方法**。

UML 基本结构

构成	内容
基本构造块	事物 Thing、关系 Relationship、图 Diagram
语义规则	name、scope、visibility、integrity、execution
通用机制	specification、adornment、common division、extensibility
四类事物	结构事物(Class/Interface/UseCase/Component/Node...)、行为事物(Interaction/StateMachine)、分组事物(Package)、注释事物(Note)
四种关系	依赖 Dependency、关联 Association、泛化 Generalization、实现 Realization

UML 4+1 视图 (高频简答)

视图	别名	作用	使用的图
用例视图	用户模型视图/场景视图	从 用户角度 看系统功能, 核心	用例图、活动图
逻辑视图	结构模型视图/静态视图	展现系统 静态结构	类图、对象图、顺序图/协作图
进程视图	行为模型视图/动态视图	描述 并发与同步 , 关注非功能性	状态图、活动图
构件视图	实现模型视图/开发视图	软件代码 静态组织	构件图
部署视图	环境模型视图/物理视图	硬件拓扑结构与软硬件映射	部署图

★评分点 "4+1" 中的 "1" = 用例视图, 是核心、驱动其他四个视图。

UML 9 种基本图

1. **用例图**: 用户角度描述系统功能
2. **类图**: 静态结构 (类及关系)
3. **对象图**: 某时刻的静态结构
4. **顺序图**: 按时间顺序的对象交互
5. **协作图**: 按对象关系的交互 (UML2.0 称通信图)
6. **状态图**: 对象状态转换条件与响应
7. **活动图**: 完成某用例的活动次序
8. **构件图**: 实现元素的封装组织
9. **部署图**: 环境元素配置, 映射实现元素

记忆: 用类对顺协状活构部 (用例、类、对象、顺序、协作、状态、活动、构件、部署)。

4.2 面向对象需求分析建模 (OOA 双模型)

OOA 包含两个核心模型:

- **领域模型**: 当前系统逻辑模型的**静态结构**及**业务流程**
- **用例模型**: 目标系统的**逻辑模型**, 定义"目标系统**做什么**"

4.3 领域模型 (Domain Model)

定义

针对某一特定领域内**概念类**或**对象**的抽象可视化表示, 又称**概念模型/分析对象模型**。表示手段:

- **UML 类图**: 表达**概念类及其关系** **业务背景**
- **UML 活动图**: 表示**业务流程**, 辅助解释类图

领域模型 ≠ 软件模型 (重要!)

- 领域模型元素是**概念类**, 不是软件类、不是软件对象、不是 DAO/Controller。
- 不适用于领域模型: 窗口、界面、数据库、有职责或方法的对象。
- OO 关键思想: 逻辑层软件类的名称源于领域模型概念类, **减小思维与软件模型的表示差异**。

识别概念类: 名词短语法

1. 从用例描述中**划出名词/名词短语**;
2. **能赋值的是属性** (数字/文本); **不能赋值的可能是概念类**;
3. **举棋不定时, 优先作为概念类处理**;
4. 不存在名词到类的精确映射机制 (自然语言二义性)。
同一个名词在不同语境里可能是概念类, 也可能是属性, 规则给不死

示例: "人民医院是北京的三甲医院" → 概念类: 医院; 属性: 名称 (人民)、地区 (北京)、级别 (三甲)。 **更多例子 教材P89**

创建领域模型 4 步

1. 找出当前需求中的**候选概念类**;
2. 在领域模型中**描述概念类**, 用问题域词汇命名, 排除无关概念;
3. 在概念类间**添加关联** (关联、继承、组合/聚合);
4. **添加必要属性**。

识别关联

- **"需要知道"型**: 关系信息须保存一段时间——**着重考虑**
- **"只需理解"型**: 增强理解, 无须保存 树木组成森林 商品放在货架上
- 3 种高优先级: **A 是 B 的一部分 (物理/逻辑)**; **A 包含/依赖 B**; **A 是对 B 的描述** 考题规格说明是对考题的描述

★评分点 识别概念类比识别关联更重要; 关联不是越多越好, 避免冗余或导出关联。

UML 类的表示

领域模型不需要操作

矩形分 3 区: 类名 (必需) / 属性 / 操作 (后两区可选)。属性格式: 可见性 属性名:类型名=初始值。可见性: +Public、-Private、#Protected、~Package。

4.4 类的六大关系（必考）

按由松散到紧密排序：

关系	语义	UML 画法	强弱
依赖 Dependency	A 使用 B 的对象（参数/局部变量/静态方法调用），临时性	带箭头虚线	最弱
关联 Association	A 的属性声明了 B 的对象引用，长期协作	无箭头实线（双向）/ 单向带箭头	弱
聚合 Aggregation	整体-部分，部分可独立存在，被多整体共享	空心菱形实线，菱形接整体	中
组合 Composition	整体-部分，部分生命周期依附整体，不可共享	实心菱形实线，菱形接整体	强
泛化/继承 Generalization	子类是父类的类型，复用属性与方法	空心三角实线，三角接父类	很强
实现 Realization	类实现接口契约	空心三角虚线，三角接接口	最强

关联的细分

- 双向关联：A 持有 B，B 持有 A（无箭头实线）
- 单向/导航关联：仅 A 持有 B（带箭头实线）
- 递归关联：自身关联
- 关联类：用虚线连接到主类之间的关联线

聚合 vs 组合（最高频）

- 聚合（空心菱形）：学校和教师（教师可独立存在）。
- 组合（实心菱形）：文档与章节（无文档则无章节）；窗口由 TextField/Button/MenuBar 组成（整体实例化部分）。

关键代码模式（判断题考点）

依赖：A 的方法参数/局部变量使用 B
class A { void f(B b){...} B b = new B(); }

关联：A 的属性持有 B
class A { private B[] b; }
class B { private A a; } // 双向

聚合：属性持有，但 B 可被外部传入
class Shipment { private Ship ship; Customer[] c; }

组合：整体在构造时 new 出部分
class Window { TextField t = new TextField(); }

4.5 用例模型（4 要素，必考）

定义
以用例为核心从使用者角度描述待构建系统的功能需求。

4 个组成部分

1. 用例图（基础）

2. 用例说明（基础）

3. 系统顺序图 SSD（附加，可省）

4. 操作契约（附加，可省）

★评分点 必考填空：用例模型 4 要素 = 用例图 + 用例说明 + 系统顺序图 + 操作契约。前两者基础，后两者附加（用例简单时可省略）。

4.5.1 用例图

3 个基本元素

- Actor（角色/参与者）：使用系统的对象，可为人、组织、外部系统、设备、时钟(Timer)。
- 用例(Use Case)：描述 Actor 如何使用系统功能实现需求目标的一组成功场景+一系列失败场景的集合。
- 关联(Association)：Actor 与用例之间、用例与子用例之间的关系。

问题域边界

矩形表示，边界名称写在矩形内或上方。Actor 画在外部，用例画在内部。

角色识别提示问题

1. 系统的主要用户是谁？谁使用系统完成日常工作？
2. 谁关注系统中记录的信息？谁来维护系统？
3. 系统与哪些其他系统交互？
4. 是否有预定时刻自动发生的事件？（→时钟）
5. 系统控制的硬件设备有哪些？

用例识别提示问题

- 系统承载的日常业务有哪些？需要哪些功能？
- 系统需要存储的数据信息？
- 各种角色能得到哪些查询、统计、报表？
- 哪些事件可以影响系统状态？

用例 3 大误区（必考判断）

1. 非目的性用例：登录/密码验证不应作为独立用例，应为某些用例的包含子用例。
2. 系统操作型用例：数据库连接、数据通信、信息获取等不应作为用例（属系统级操作，非用户视角）。
3. 粒度过细：增删改查应合并为"XX 管理"，不要拆成 4 个独立用例。

4.5.2 基本用例与子用例

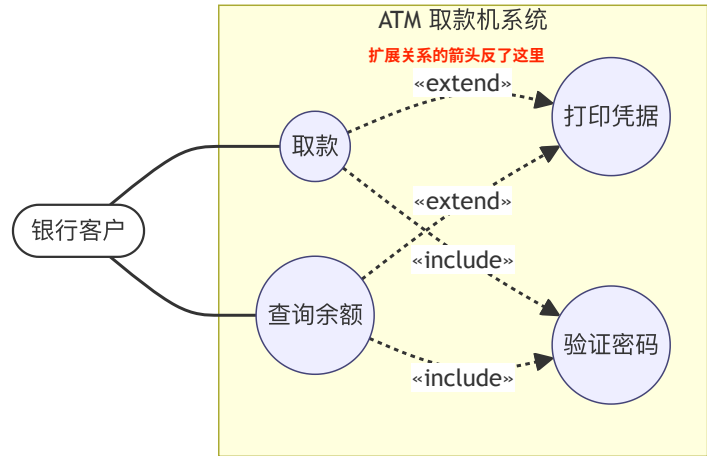
分类	定义	触发条件
基本用例	与角色直接相关	由 Actor 启动
包含子用例	多个基本用例共有的必须执行步骤被抽出 e.g. 登录/注册	无条件，必调用
扩展子用例	条件判断分支被独立出来	仅当某条件成立才调用

4.5.3 包含 <> vs 扩展 <>

对比项	包含 <>	扩展 <>
触发	必定执行	条件触发
用途	复用相同步骤	处理条件分支/可选/异常
典型场景	多个用例共有的登录验证	VIP 支付、打印凭据、忌口选择
是否修改原用例	需修改原用例插入调用点	不修改原用例
箭头方向	基用例 → 被包含用例	扩展用例 → 基用例
子用例独立性	可作为独立功能（一般由 Actor 直接或间接驱动）	不直接由 Actor 启动

★评分点 考点速记：include 必发 → 基指向子；extend 条件发 → 子指向基。"不能修改原用例文本"时用 extend。

用例图示例：ATM 取款（重绘）



说明：验证密码为包含子用例（必发）；打印凭据为扩展子用例（条件可选）。

4.5.4 用例说明（用例描述）

字段	含义
用例编号	唯一编号，便于索引
用例名称	(状语+)动词+(定语+)宾语，体现参与者目标
范围	软件系统的作用范围
级别	企业目标/用户目标/子系统目标级别
参与者	主要参与者
项目相关人员及兴趣	满足所有相关人员兴趣
前置条件	场景开始前必须为真的条件
后置条件	用例成功结束后必须为真的条件
主要成功场景	典型成功路径，1..n 步
扩展(替代流程)	*a 任意时刻/5a 第5步分支
特殊需求	非功能性需求
技术与数据变化列表	IO 方式与数据格式
发生频率	用例执行频率

4.5.5 系统顺序图（SSD）

定义

SSD 通过 UML 交互图元素，描述一个用例中角色与代表系统的对象之间某一场景的消息交互形式。

SSD 3 类基本元素（重点）

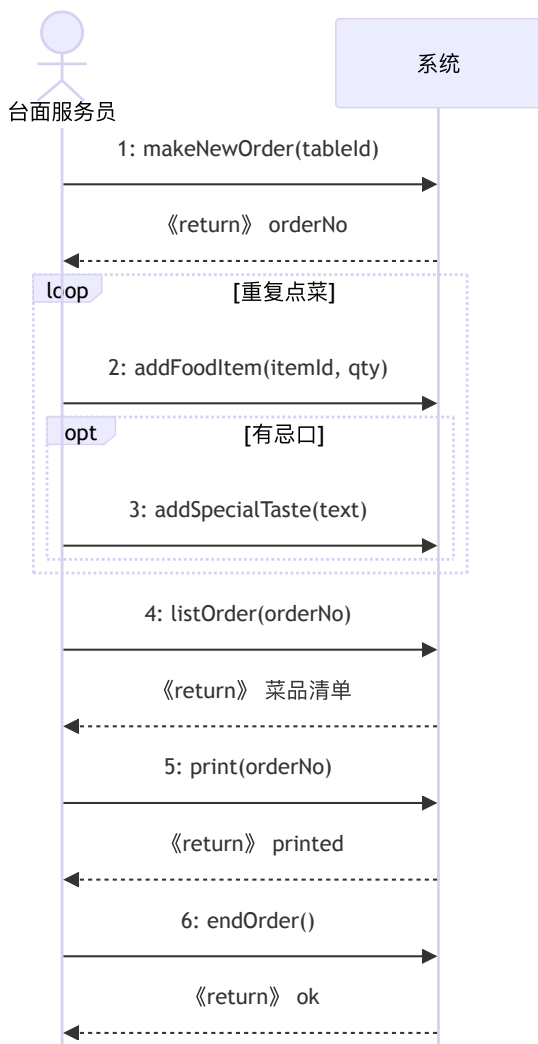
- 角色（左侧生命线）
- 代表软件系统的对象（右侧生命线，唯一）一般就是 System
- 角色与系统之间的交互信息（消息/操作）

★评分点 SSD 只有两类对象：Actor + 系统（system）。某些特殊场景可有第 3 个 Actor，但系统对象只能 1 个。角色把系统视为黑盒。

消息类型

- 同步消息：消息名(参数1, 参数2,...)
- 返回消息：《return》或 《return》 X,Y
- 循环：loop 片段
- 可选：opt 片段

SSD 示例：餐馆订单处理



4.5.6 操作契约 (Operation Contract)

定义

系统操作：处理系统事件的操作。操作契约是**为系统操作定义的**，参考领域模型，描述系统事件完成后业务对象的状态及操作执行结果，以便软件设计参考。

4 项结构

项	含义
操作	操作名称及其参数
交叉引用	产生该操作的用例（可选）
前置条件	执行操作前系统或领域对象的状态
后置条件	执行操作后领域模型对象的状态（必考三要素）

★评分点 后置条件三要素（必考）：

1. **对象的创建或消除**；
2. **对象之间"关联"的建立或消除**；
3. **对象的属性值修改**。

后置条件是**声明性**的，强调"状态变化"，不强调"如何实现"。

编写步骤

1. 根据 **SSD 识别所有系统事件**（操作）；
2. 针对每个系统操作结合用例领域模型，找到**相关概念类对象**；
3. 对相对复杂/用例描述不清楚的系统操作创建操作契约；
4. 按三要素描述后置条件。

SSD 与操作契约的关系（必考简答）

- 一一对应：**SSD 中每条角色→系统的同步消息对应一个操作契约**。
- 派生关系：操作契约基于 SSD 编写——SSD 给"消息形状"，契约给"副作用（状态变化）"。
- 关注点互补：SSD 关注消息名+参数+返回；契约关注领域对象状态变化。
- 同属用例模型：均为用例模型的 4 要素之一。

示例：MakeNewOrder(TableId)

系统事件	MakeNewOrder(TableId)
交叉引用	订单处理
前置条件	服务员身份验证通过，开始订单处理
后置条件	① 创建一个新的(概念类)订单实例； ② 订单与(概念类)台面建立关联； ③ 订单与(概念类)台面服务员建立关联； ④ 订单属性初始化：订单流水号、订单时间、存储菜品的数组

示例：AddFoodItem(ItemId, quantity)

系统事件	AddFoodItem(ItemId, quantity)
交叉引用	订单处理
前置条件	服务员正在处理订单
后置条件	① 创建一个新的(概念类)菜品实例； ② 菜品与订单建立关联； ③ 订单与菜品描述建立关联； ④ 菜品属性被修改：quantity

⑤ 补充知识点（教材独有）

OOA 经典方法演进

方法	提出者	关键点
OOA/OOD	Coad & Yourdon 1991	OOA 5 步（类对象→结构→主题→属性→服务），5 层模型；OOD 4 部分（问题域/人机交互/任务管理/数据管理）
Booch	Grady Booch	强调迭代与创造力；4 主图（类图/对象图/模块图/进程图）+ 2 辅图（状态转换/时序）
OMT	Rumbaugh 1991	3 模型（对象/动态/功能）；4 步（分析/系统设计/对象设计/实现）
OOSE	Jacobson	5 模型（需求/分析/设计/实现/测试）；首次使用 Use Case

UML 是综合 Booch + OMT + OOSE 的演化结果。

UML 发展 4 阶段

1. 独立发展（80 年代~1993，百家争鸣）
2. 合并统一（1994~10 起，Booch + Rumbaugh + Jacobson 加入 Rational）
3. 标准化（1997 UML 1.1 被 OMG 采纳）
4. 工业界应用（1998 起，OMG 接管，事实标准；最新 2.0）

UML 目标

- 提供现成、易用、表达能力强的可视化建模语言
- 提供可扩展机制（约束 Constraint、版型 Stereotype、标记值 Tagged Value）
- 与具体实现、过程无关
- 支持构件、协作、框架、模式等高级概念

识别属性的指导原则

- 优先定义为简单数据类型（Boolean/date/number/string/text/time/address/color/zip 等）
- 属性值不应是对象——联系概念类应该用关联而非属性
- 只考虑与应用相关的属性，避免因实现而引入属性
- 建模时只需属性名、类型、初始值（可见性推迟到设计阶段）

常见关联列表（高优先级 3 项）

1. A 在物理上或逻辑上是 B 的一部分（树木—森林）
2. A 在物理上或逻辑上包含在 B 中或依赖 B（商品—货架）
3. A 是对 B 的描述（考题规格说明—考题）

SSD 中循环与可选

- loop 片段：重复步骤（如循环记录每道菜）
- opt 片段：可选分支（如客人有忌口时）

用例描述完整示例（餐馆订单处理）

用例编号	Uc_006
用例名称	订单处理
范围	餐馆信息管理系统
级别	用户目标级别
参与者	台面服务员
前置条件	顾客落座且请求点菜
后置条件	生成该台面的订单
主要成功场景	1. 顾客请求点菜 2. 服务员使用 PDA 开始点菜 3. 包含用例: 登录系统 4. 服务员输入台面号 5. 记录菜品，如有忌口则转 扩展用例: 忌口选择 6. 重复 5 直至点菜完毕 7. 复述所点菜品确认 8. 顾客确认后打印订单，包含用例: 打印订单 9. 点菜服务完成
扩展	*a. 系统任意时刻失败 5a. 忌口选择：辣、咸、葱、姜、蒜等
特殊需求	3 秒内反应；使用无线 PDA

需求分析规格说明书

OOA 的最终产物是需求规格说明书，由领域模型 + 用例模型两个核心部分组成。具体模板参见附录一。

⑥ 核心对比表

对比 1：包含 <> vs 扩展 <>

对比项	包含 (Include)	扩展 (Extend)
核心特征	必发	条件触发
原用例修改	需修改插入调用	不修改原用例
箭头	基 → 子（虚线）	子 → 基（虚线）
用途	抽取公共步骤	抽取条件分支
子用例独立	是	否
典型	密码验证、登录、打印订单	VIP 支付、忌口选择、积分换里程

对比 2：聚合 vs 组合

对比项	聚合 Aggregation	组合 Composition
菱形	空心	实心
部分独立性	可独立存在，被多整体共享	生命周期依附整体
实例化方	外部传入	整体 new 出
代码特征	private Ship ship; (外部传入)	TextField txt = new TextField();
典型	学校—教师；船运—船只	文档—章节；窗口—控件

对比 3：关联 vs 依赖

对比项	关联 Association	依赖 Dependency
线型	实线（可带箭头）	虚线带箭头
耦合	长期、属性持有	临时使用（参数/局部/静态调用）
代码	class A { private B b; }	class A { void f(B b){} }
强度	较强	最弱

对比 4：领域模型 vs 软件模型

对比项	领域模型	软件模型（设计模型）
元素	概念类（业务世界名词）	软件类/对象（解域）
关注	"有什么"（问题域）	"怎么做"（解域）
方法	不写方法	有职责/方法
典型	病人、医生、订单、商品	Controller、DAO、DBFacade、UI
产物阶段	OOA（分析）	OOD（设计）

对比 5：SSD vs 设计模型顺序图

对比项	系统顺序图 SSD	设计模型顺序图
章节	第 4 章（OOA）	第 7 章（OOD）
对象数量	2 类（Actor + 系统）	多类（Controller/Domain/DAO/DB 等）
系统可见性	黑盒	白盒
关注	系统事件+返回	对象协作+方法调用
触发关键词	系统顺序图、SSD、系统事件、操作契约	GRASP、控制器、信息专家、设计模型、DAO/DBFacade

对比 6：OOA vs OOD

对比项	OOA 分析	OOD 设计
目标	问题域理解，建 OOA 模型	解域实现，建设计模型
产物	领域模型 + 用例模型	设计类图 + 交互图 + 体系结构
是否考虑实现	否	是
表示法关系	OOA 与 OOD 采用一致的面向对象概念和 UML 表示法，无缝衔接，无结构化方法的鸿沟	

⑦ 开放题答题模板

- **关系**：概念类是软件类的**输入与启发**（命名+属性来源于领域模型），软件类进一步被分配行为型职责（方法）并遵循 GRASP。

模板 1：如何构建领域模型

1. **明确范围**：确定当前迭代与问题域范围。
2. **识别概念类**：用**名词短语**从用例描述中提取名词，能赋值的归为属性，不能赋值的作概念类；举棋不定优先作概念类。
3. **建立关联**：识别"需要知道"型关联（含整体-部分、包含、描述、交易等），用实线连接并写关联名称。
4. **添加基数**：1、*、0..1、1..* 等。
5. **识别特殊关系**：继承（空心三角）、聚合（空心菱形）、组合（实心菱形）。
6. **添加属性**：简单数据类型（数字、文本、日期），属性值不应是对象。
7. **排除软件类**：不画 Controller、DAO、DB、UI。

模板 2：如何编写操作契约

1. **识别系统事件**：从 SSD 抽取所有 Actor → 系统的同步消息作为操作。
2. **定位概念类**：结合领域模型找到相关概念类对象。
3. **写前置条件**：操作执行前系统或领域对象的状态。
4. **写后置条件**（三要素必写）：
 - 对象创建/消除
 - 关联建立/消除
 - 属性值修改
5. **声明性**：描述状态变化，不写算法流程。

模板 3：用例模型包含哪些？作用？

1. **4 要素**：用例图、用例说明、系统顺序图、操作契约（前两者基础，后两者附加）。
2. **作用**：从**用户视角**描述系统功能需求；以用例为驱动贯穿分析-设计-测试；为软件对象设计提供充分信息。
3. **与领域模型关系**：领域模型"有什么"（静态），用例模型"做什么"（动态行为），二者相辅相成、相互校验，共同构成 OOA 双视图。

模板 4：操作契约主要描述什么状态变化？

系统操作完成后，**领域模型中对象的状态变化**（即后置条件）：

1. **对象的创建或消除**（实例化/删除）；
2. **对象之间"关联"的形成或断开**（链 link 建立/消除）；
3. **对象的属性值修改**。

模板 5：用例方法为何优于功能特性列表？

1. 用例将系统功能放到**面向用户目标的语境**中，确保识别出来的功能真正为用户提供价值；
2. 用例以**多种场景**（成功+失败）描述，与用户日常工作序列一致，便于用户-开发者交流；
3. 用例可驱动后续分析、设计、测试全生命周期。

模板 6：SSD 中的指令与操作契约的关系

1. **一一对应**：每条 SSD 同步消息 ↔ 一个操作契约。
2. **派生关系**：契约基于 SSD 编写——SSD 识别系统事件，契约定后置条件。
3. **关注点互补**：SSD 关注"消息形状"（名称+参数+返回）；契约关注"领域对象状态副作用"。
4. **同属用例模型**：均为 4 要素之一。

模板 7：领域模型、用例模型、设计模型的图使用

模型	使用的 UML 图
领域模型	类图（核心）+ 活动图（业务流程辅助）
用例模型	用例图 + 用例描述 + 系统顺序图 + 操作契约
设计模型	动态：交互图（顺序图/协作图）；静态：设计类图、包图、组件图

领域模型 → 用例模型（提供业务概念）→ 设计模型（动态结构推导静态结构），三者迭代精化。

模板 8：操作契约对象 vs 软件对象的区别

- **后置条件对象**：领域模型中的**概念类对象**，是分析阶段的业务抽象，不涉及实现。
- **软件对象**：OOD 阶段产物，可实例化的设计类，含技术职责（持久化、事务等）。

⑧ 历年真题汇总与详解（按题型分类）

一、判断题

2010–2011 期中 A · 判断 6

面向对象分析(OOA)和面向对象设计(OD)分别采用不同的概念和表示法。

原答案 × (错误)

解析：教材 §4.1.1、§7.2.2 明确 OOA 与 OD 采用一致的面向对象概念（类、对象、属性、操作、关联、继承、聚合等）和统一的 UML 表示法。OD 的问题域设计部分和 OOA 并没有严格分界线，分析与设计之间无缝衔接，反映开发活动的本质。

2010–2011 期中 A · 判断 8

用例模型中，创建系统操作契约是必须的。

× (错误)

解析：教材 §4.5.2 明确：用例模型 4 要素中，用例图 and 用例说明是基础，SSD 和操作契约是附加说明。当某个用例比较简单时，可以省略 SSD 和操作契约。

2007–2008 期末 A · 判断 2

用例模型是用来说明系统应该具备的功能描述。

√ (正确)

解析：教材 §4.5.1：用例模型以用例为核心从使用者的角度描述和解释对于待构建软件系统的功能需求。

2007–2008 期末 A · 判断 4

UML 中顺序图和协作图不仅能用来表示对象之间的动态行为，也能表示对象内部的状态变化。

× (错误)

解析：顺序图、协作图（UML2.0 称通信图）均属交互图，描述对象之间的消息交互。描述对象内部状态变化的是状态图（State Diagram）。

2007–2008 期末 A · 判断 8

领域模型是面向对象分析和设计的一个组成部分，因而它也是待构建的软件模型的一个部分。

× (错误)

解析：教材 §4.4.3 明确：领域模型描述的内容与软件对象无关，是纯粹对现实客观世界的抽象描述。其元素是概念类而非软件类，仅为软件设计提供启发，不属于待构建的软件模型本身。

2007–2008 期末 A · 判断 9

在顺序图中，一个对象 A 发送了一条创建另一个对象 B 的消息，那么表明对象 B 具备了处理该条消息的职责。

√ (正确)

解析：UML 顺序图中接收消息的对象具有处理该消息的职责和能力（GRASP 信息专家模式基础）。A 向 B 发送 «create» 消息，意味着 B 类需要提供相应的构造方法。

2010–2011 期中 B · 判断 8

用例模型中，操作契约是描述软件如何实现用例的过程。

× (错误)

解析：操作契约是"系统对象根据需求的具体内容返回一个明确的结果"，即契约性结果（后置条件），不是描述"实现过程"。描述实现过程的是用例描述和 SSD。

2010–2011 期末 B · 判断 6

在顺序图中，对象 A 发送给了对象 B 一条消息，那么表明对象 A 具备了处理该条消息的职责。

× (错误)

解析：顺序图中接收消息的对象 B 才具有处理该消息的职责，A 仅是发起方。

2010–2011 期末 B · 判断 9

操作契约是为系统操作定义的，描述系统操作执行的过程。

× (错误)

解析：操作契约描述的是系统操作执行后领域对象的状态变化（后置条件三要素），不是执行过程。过程由交互图描述。

2011–2012 期末 B · 判断 2

面向对象分析(OOA)和面向对象设计(OD)分别采用不同的概念和表示法。

× (错误)

解析：教材 §7.2.2：OOA 与 OD 采用一致的表示法，两者能紧密衔接，大大降低过渡难度、工作量和出错率。

2011–2012 期末 B · 判断 10

用例控制器不需要实现系统操作，但外观控制器需要。

× (错误)

解析：教材 §7.3.2：不论是外观控制器还是用例控制器，它们只是接收系统事件消息，并没有实现系统事件请求的内容。实际处理由业务逻辑层对象完成。

2015–2016 期中 A · 判断 8

面向对象的需求分析建模方法包括领域模型和用例模型。

√ (正确)

解析：教材 §4.3 明确：OOA 建模的两个核心活动即领域建模和用例建模，二者相辅相成。

2018–2019 期末 B · 判断 8

无论外观还是用例控制器，均无需实现系统事件请求的内容。

√ (正确)

解析：教材 §7.3.2 原文：不论是外观控制器还是用例控制器，它们只是接收系统事件消息，并没有实现系统事件请求的内容。

2024–2025 期中 A · 判断 9

业务流程可以通过 UML 的用例图进行表达。

× (错误)

解析：教材 §4.4.2：领域模型使用类图表达概念，使用活动图表示业务流程。用例图描述角色对系统的功能需求，不表达流程次序。

2024–2025 期中 A · 判断 10

操作契约的后置条件描述的是系统顺序图中的返回消息。

× (错误)

解析：后置条件描述领域模型对象的状态变化（对象创建/消除、关联、属性修改），与 SSD 中的《return》返回消息不同。

二、单选题

2010–2011 期中 A · 单选 3 / 2008–2009 期末 A · 单选 9

OOA 所要完成的工作不包括 ()。

A. 建立用例模型 B. 建立领域模型 C. 建立操作契约 D. 定义完善的类的属性和操作

D

解析：OOA 核心产物是用例模型、领域模型、(可选) SSD 与操作契约 (教材 §4.4、§4.5)。“定义完善的类的属性和操作”属于 OOD 阶段对设计类的细化，不属于 OOA。

2010–2011 期中 A · 单选 5

如果由于某种原因不能修改已有的用例文本，使用以下哪种关系可以解决这个问题 ()。

A. 包含关系 B. 继承关系 C. 扩展关系 D. 聚合关系

C

解析：教材 §4.5.4：包含需修改原用例插入调用点；**扩展关系通过条件成立时被调用，对基础用例文本无侵入性修改。**

2010–2011 期中 B · 单选 5

如果子用例是基本用例的一部分，使用以下哪种关系可以表示这个关系 ()。

A. 包含关系 B. 继承关系 C. 扩展关系 D. 聚合关系

A

解析：教材 §4.5.4：包含关系指“一段或几段相似的操作步骤……可表示为一个独立的系统功能”，即子用例作为基本用例必不可少的一部分被显式调用。

2009–2010 期末 A · 单选 4

系统顺序图属于哪一个模型 ()。

A. 用例模型 B. 分析模型 C. 动态结构模型 D. 设计模型

修正答案：A (用例模型)

解析 (按教材修正)：教材 §4.5.2 明确把“系统顺序图 (SSD)”作为用例模型的组成之一。SSD 是第 4 章 OOA 阶段的产物，描述 Actor 与系统黑盒的消息交互；第 7 章设计模型中的“设计顺序图”才是描述软件对象协作的图。**题库答案“设计模型”不符合本课程第 4 章口径，按教材应选 A。**

2010–2011 期末 B · 单选 8

操作契约属于哪一个模型 ()。

A. 用例模型 B. 测试模型 (C 项缺失) D. 设计模型

A

解析：教材 §4.5：操作契约是用例模型的附加组成部分，与 SSD 一起为软件对象设计提供依据。

2011–2012 期末 B · 单选 4

UML 顺序图可以表示以下什么模型 ()。

A. 用例模型 B. 领域模型 C. 设计模型 D. 用例模型+设计模型

D

解析：**顺序图既作为用例模型的 SSD (系统顺序图，第 4 章)，又作为设计模型的动态结构 (设计顺序图，第 7 章)。**

2013–2014 期末 A · 单选 2

如果用例之间存在扩展关系，则下面哪一个描述正确 ()。

A. 基本用例指向子用例 B. 子用例在某种条件下指向基本用例 C. 基本用例在某种条件下指向子用例 D. 双方互指

B

解析：教材 §4.5.4 + UML 规范：扩展关系由**扩展用例 (子用例)**通过 «extend» 依赖**指向基本用例**，在基本用例扩展点条件成立时触发。

2018–2019 期末 B · 单选 3

以下哪一项不能作为用例图中的角色 ()。

A. 人 B. 其他系统或设备 C. 数据 D. 时钟

C

解析：教材 §4.5.3：角色可分为人、其他系统或设备、时钟 (Timer)。**数据是被动信息**，不具有主动交互能力，不能作为角色。

2018–2019 期末 B · 单选 8

面向对象的需求分析方法中，在操作契约的后置条件部分需要描述并确定领域模型中对象的状态变化，以下哪个选项不属于后置条件中要求的内容 ()。

A. 对象创建或者消除 B. 对象之间的“关联”创建或者消除 C. 确定与本操作相关的概念类对象 D. 对象的属性值修改

C

解析：教材 §4.5 操作契约表 4–8 明确：后置条件 3 类为①②③ 对象创建/消除、关联、属性修改。“**确定相关概念类对象**”属于领域建模识别工作 (§4.4 阶段)，不属于后置条件。

2024–2025 期中 A · 单选 6

用例 A 是用例 B 和用例 C 在某种条件成立时可以执行的过程，它们之间的关系是 ()。

A. 包含关系 B. 继承关系 C. 聚合关系 D. 扩展关系

D

解析：教材 §4.5.4：“在某种条件成立时可以执行”正是**扩展关系**的典型特征。

2024–2025 期中 A · 单选 7

下面哪一项不是用例模型中操作契约所关注的内容 ()。

A. 属性的修改 B. 实例的创建 C. 方法的调用 D. 关联的形成

C

解析：操作契约后置条件 3 类对应 A、B、D。“**方法的调用**”属软件对象设计细节，非操作契约关注。

2024–2025 期中 A · 单选 9

对于“用例”描述错误的是 ()。

A. 一个用例可以由多个角色驱动 B. 用例是参与者使用系统一组成功场景和失败场景的集合 C. 用例描述的功能粒度越小越好 D. 用例中只描述参与者可以看到的系统行为特征

C

解析：教材 §4.5.4 “常见误区三：用例表示的粒度”明确反对将用例划分过细 (如表单增删改查拆为 4 个)。“**功能粒度越小越好**”错误。

2023–2024 期末 A · 单选 2

下面哪一项不属于操作契约中后置条件所需规定的内容 ()。

A. 对象被创建或者消除 B. 对象之间的“关联”被创建或者消除 C. 对象中的功能被定义 D. 对象的属性值被修改

C

解析：后置条件仅描述状态变化 (A、B、D)。“**功能被定义**”属职责分配 (OOD 信息专家等模式)。

2023–2024 期末 A · 单选 3

下面哪一项属于领域模型的范畴 ()。

A. 用例图 B. 系统顺序图 C. 活动图 D. 操作契约

C

解析：教材 §4.4.2：领域模型用**类图**表达概念，**活动图**表示业务流程。A、B、D 均属用例模型。

三、填空题

2023–2024 期末 A · 填空 3

用例模型由四个部分构成：用例图、用例说明、系统顺序图和（ ）。

操作契约 (Operation Contract)

解析：教材 §4.5.2。前两者基础，后两者附加。

2024–2025 期中 A · 填空 6 (与第 2 章交叉)

UP 模型的特点可以概括为以用例为驱动、() 为核心的应用 () 的生命周期模型。

架构；迭代增量 (式)

解析：RUP 三大特点：用例为驱动、以架构为核心、迭代增量式开发模型。

四、简答题

2009–2010 期末 A · 简答 1

用例模型是什么？其主要构成有哪几个？并描述其作用。

答：

(1) 定义：用例模型以用例为核心从使用者角度描述和解释待构建软件系统的功能需求。

(2) 主要构成 4 要素：

1. 用例图：Actor + 用例 + 关联
2. 用例说明 (描述)：成功场景 + 扩展场景
3. 系统顺序图 SSD (附加)：Actor 与系统黑盒的消息交互
4. 操作契约 (附加)：系统事件的后置条件

(3) 作用：捕获和明确功能需求；为需求确认、合同签订、工作量估算提供依据；以用例为线索贯穿分析–设计–测试全生命周期。

2009–2010 期末 A · 简答 2

面向对象的领域模型是什么？其主要作用是什么？并描述它与用例模型的关系。

答：

(1) 定义：领域模型是对问题域中客观存在的概念类 (实体) 及其属性、关联的结构化抽象，常用 UML 类图表达，又称概念模型。关注“有什么”。

(2) 主要作用：

1. 建立问题域的统一词汇表；
2. 揭示概念类之间的关联、聚合、继承关系；
3. 为分析、设计阶段提供对象雏形；
4. 辅助需求验证。

(3) 与用例模型的关系：

- 互补：领域模型从内部结构视角 (静态)，用例模型从外部行为视角 (动态)，一内一外、一静一动；
- 相互校验：用例描述中的名词→概念类候选，动词→关联/方法候选；
- 共同演化：迭代增量式相互精化。

2007–2008 期末 A · 简答 1 / 2011–2012 期末 B · 简答 1

简述面向对象开发方法中 OOA 和 OOD 要完成的工作。

OOA 工作 (建立领域模型 + 用例模型)：

1. 识别概念类：用名词短语法从问题域中识别；
2. 建立领域模型：UML 类图描述概念类及关系 (关联、聚合/组合、继承、依赖)，活动图描述业务流程；与软件对象无关；
3. 用例建模：识别 Actor 与用例，画用例图，写用例描述，必要时补 SSD 与操作契约；
4. Coad/Yourdon 经典 OOA 5 层：主题层、类与对象层、结构层、属性层、服务层。

OOD 工作 (转换为软件设计模型)：

1. 软件体系结构设计 (分层/MVC)；
2. 用例实现：用 GRASP 模式分配职责，绘制交互图；
3. 设计类图：确定每个软件类的方法、属性、关系；
4. UI 设计、持久化设计、状态迁移图；
5. Coad/Yourdon 经典 OOD 4 部分：问题域、人机交互、任务管理、数据管理。

OOA vs OOD 优势：采用一致的面向对象概念和 UML 表示法，无缝衔接，相比结构化方法避免了分析与设计的鸿沟。

2010–2011 期中 B · 简答 2

简述用例模型、领域模型及设计模型之间各种图的使用关系。
答：

模型	使用的 UML 图
领域模型	类图（概念类及关系）+ 活动图（业务流程辅助）
用例模型	用例图 + 用例描述 + 系统顺序图 + 操作契约
设计模型	动态：交互图（顺序图/协作图）；静态：设计类图、组件图、包图

关系：领域模型 →（提供业务概念）→ 用例模型 →（系统事件驱动）→ 设计模型动态结构 →（推导）→ 设计模型静态结构；操作契约与设计过程可能反过来修订领域模型。

2013–2014 期末 A · 简答 2

什么是用例？为什么用例方法比功能特性列表更有效？（4 分）
答：

（1）定义：用例是从使用系统的角色出发描述使用者与系统之间交互的场景，即一组成功场景和一系列失败场景的集合，每个用例必须返回明确的交互结果。

（2）用例更有效的理由：

- 用例将功能放到面向用户目标的语境中，识别出来的功能真正为用户提供价值；
- 用例以多种场景（成功+失败）描述，与用户日常工作序列一致，用户–开发者交流更简单有效。

2013–2014 期末 A · 简答 4 / 2018–2019 期末 B 同类

操作契约主要描述系统操作完成之后领域模型中对象状态的变化，请问这些变化包含哪些方面？（4 分）

答（后置条件三要素）：

- 对象的创建或消除（实例化/删除）；
- 对象之间"关联"（链）的形成或断开；
- 对象的属性值修改。

2021–2022 期末 A · 简答 3

软件设计的两个阶段是如何区分的？请说明用例模型中操作契约的后置条件所表示的对象与软件设计中的软件对象的区别与关系。

答 1：设计两阶段

维度	概要设计 HLD	详细设计 LLD
目标	总体框架（功能/数据/行为模型）	各模块内部结构与算法
关键活动	体系结构、数据、处理方式设计	模块算法、局部数据结构
输出	概要设计说明书	详细设计说明书
层次	模块划分、接口（What/Where）	接近源代码（How）

答 2：契约后置条件对象 vs 软件对象

- 后置条件对象：领域模型中的概念类对象（分析阶段产物），描述业务状态变化；
- 软件对象：OOD 阶段的设计类，含技术职责（持久化、事务等）；
- 关系：概念类是软件类的输入与启发，软件类进一步被分配行为型职责并遵循 GRASP。一对一或多对一，并非完全等价。

2024–2025 期中 A · 简答 3

请说明用例模型中系统顺序图中的指令与操作契约的关系。（5 分）

答：

- 一一对应：SSD 中每条由 Actor 发送给系统的同步消息（系统事件/操作）都对应一个操作契约。
- 派生关系：操作契约基于 SSD 编写——第一步根据 SSD 识别系统事件；第二步结合领域模型找相关概念类；第三步写契约。
- 关注点互补：SSD 关注"消息形状"（名+参数+返回），契约关注"领域对象状态副作用"。
- 目的不同：SSD 明确交互顺序，契约为软件对象设计提供依据。
- 同属用例模型：均为 4 要素之一，简单用例可省略。

五、综合应用大题

2024-2025 期中 A · 综合 1 (食堂一卡通自助系统)

场景摘要：用户登录后使用余额查询/详单打印/充值；扩展"手机充值"（银行卡绑定）。

问题 1：画用例图，区分基用例、包含用例、扩展用例（8 分）

说明：登录为包含子用例（3 个基用例都需登录验证）；手机充值为扩展子用例（条件：银行卡与手机绑定）；打印收据为手机充值内部的包含步骤。

2010-2011 期中 A · 综合五 (电子商务网站)

场景摘要：客户登录→查询商品→购买→网上银行付款→输入网银账号+身份证号+密码→生成确认码→输入动态密码→完成。VIP 支付卡为扩展（八折优惠）。

问题一：用例图（区分基/包含/扩展）

包含说明：查询商品和网银付款都包含"填写身份证号和密码"；网银付款包含生成确认码和输入动态密码。扩展说明：VIP 支付卡付款扩展网银付款，扩展条件是"客户是 VIP 客户且选择 VIP 支付卡"，VIP 客户继承客户。

2009-2010 期末 A · 综合五 (网上订票订座)

场景摘要：顾客主页进入→在线订票→选影片→选场次→选座→输手机号→确认→系统短信返回 4 位确认码→完成。

问题一：用例图

2008-2009 期末 A · 综合五 (医院就诊管理系统)

场景摘要：医生使用问诊子系统→查看排队→叫号→病人提交挂号单→确认→调取以往病历→书写电子病历→开药→打印处方。

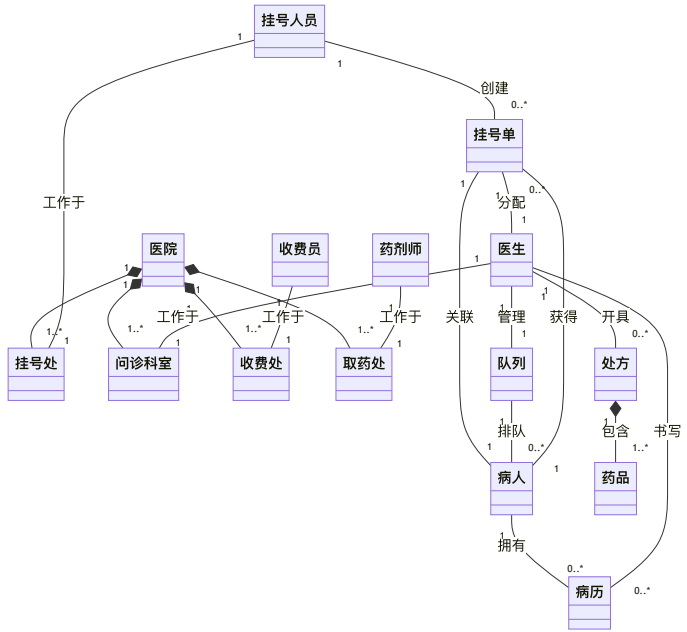
问题一（5 分）：用例模型 + 查看病历的用例说明

用例名称	查看病历
参与者	医生
前置条件	医生已确认病人挂号单
后置条件	系统返回病人历史病历信息
主要成功场景	1. 医生提交病人挂号单/病历号 2. 系统检索并显示病人历史病历 3. 医生查看既往诊断与处方
扩展	2a. 病人无历史病历：系统提示"新病人，无病历" 2b. 网络异常：系统提示稍后重试

2015–2016 期中 A · 综合（医院就诊管理系统四部门）

场景：挂号→问诊→交费→取药 4 个部门联动。

问题 1：领域模型（10 分）

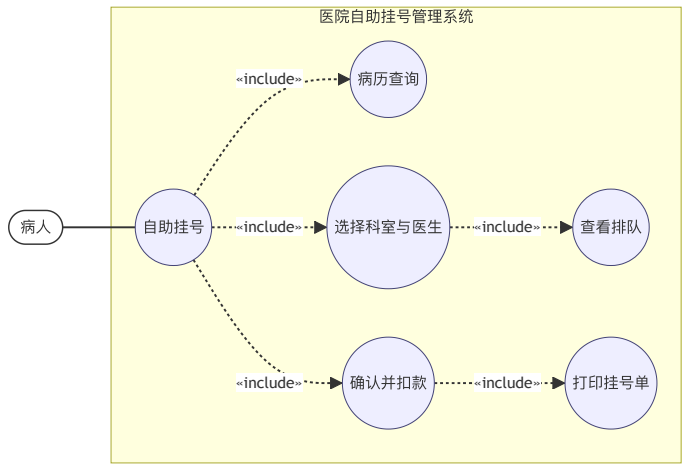


关键：医院组合 4 个部门；4 种角色工作于对应部门；挂号单关联病人与医生；处方组合药品；病历关联病人与医生。

2010–2011 期中 B · 综合 / 2010–2011 期末 B · 综合（医院自助挂号机）

场景：病人持病历卡自助挂号→输病历号→系统返回病人信息→选科室→系统给医生排队→病人选医生→系统返回挂号费→病人确认→扣款→打印挂号单。

问题 1：用例图（10 分）



说明：所有步骤由"自助挂号"基用例包含。"查看排队"作为子用例独立抽出（供选医生参考），仍属包含关系（必发）。

2007–2008 期末 A · 综合五（医院挂号和问诊管理系统）

SSD 4 条已知消息：①返回挂号单流水号/时间/工号；②根据病历号返回病人姓名/年龄/性别；③提供科室、医生、支付策略，返回挂号费；④病人支付，返回找赎+打印挂号单。

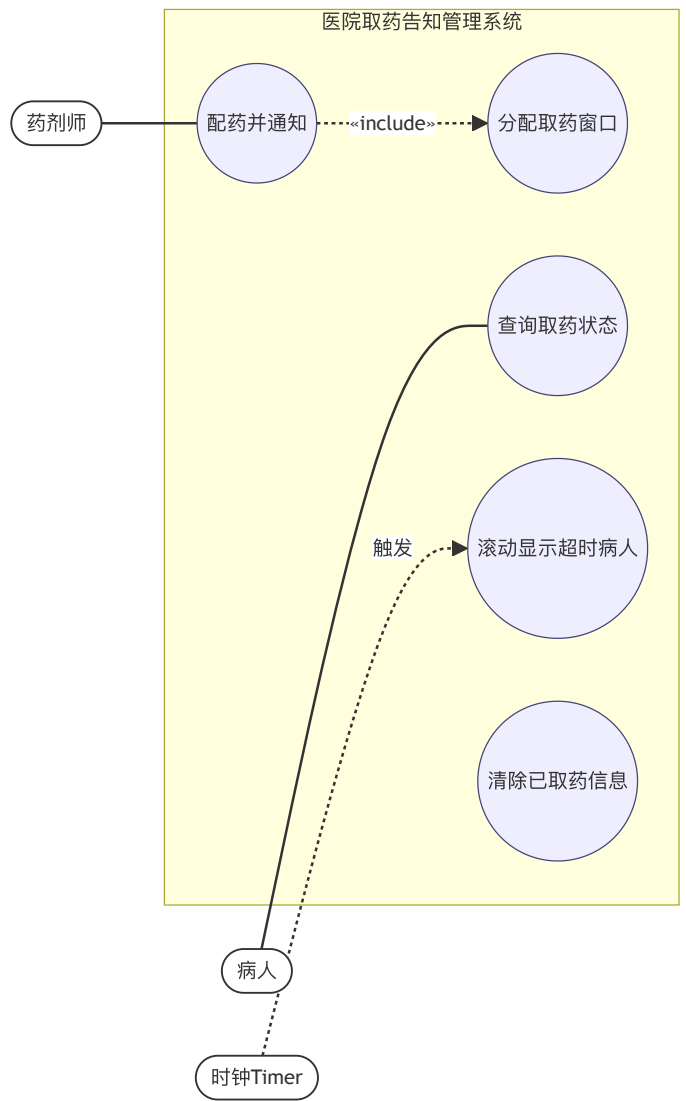
问题（典型）：根据 SSD 写操作契约

操作	前置条件	后置条件
createRegistration(...)	挂号员已开始挂号	①创建挂号单实例；②挂号单与挂号员建立关联；③挂号单属性赋值（流水号、时间）
queryPatientInfo(recordNo)	病人已注册病历	无对象创建；挂号单与病历/病人建立关联；查询缓存属性更新
selectDoctor(dept, doctor, strategy)	病人基本信息已确认	挂号单与医生/科室建立关联；挂号单.fee 被赋值；挂号单.strategy 被赋值
pay(amount)	挂号费已确定	挂号单.isPaid 被修改为 True；挂号单.change 被赋值；触发打印（isPrinted=True）

2018–2019 期末 B · 综合（医院取药告知管理系统）

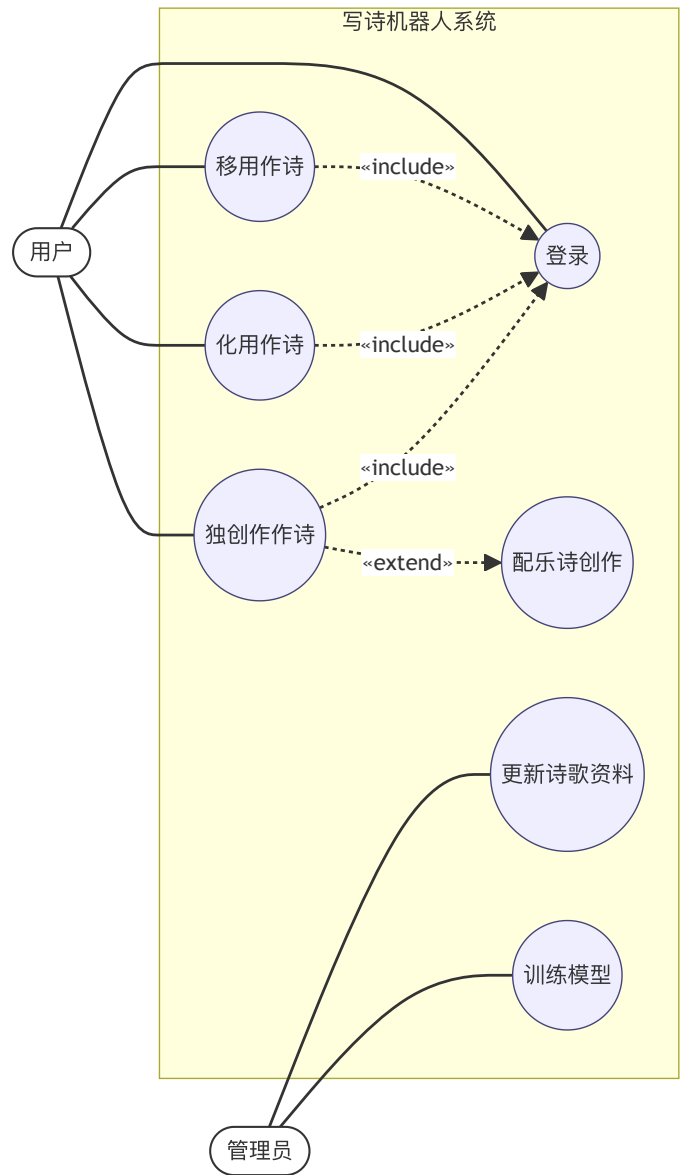
问题 1（7 分）：确定角色及用例图

Actor：药剂师、病人（就医卡扫描者）、时钟（2 分钟超时触发滚动显示）、合作硬件（大屏幕、查询机）。



说明：时钟作为特殊角色触发"滚动显示"扩展；病人用就医卡查询；药剂师配药完成后通知。

2021-2022 期末 A · 综合（写诗机器人"小红"）
场景：3 子系统（翼然/泻玉/沁芳）+ 管理员 + 扩展"配乐诗创作"。
问题 1：用例图



说明：登录为包含子用例；配乐诗创作为独创作诗的扩展（条件：用户选择配乐选项）。

2020 随堂作业（亡者农药游戏自助系统）
场景：用户登录后使用 3 个功能（游戏开始/查询战绩/好友管理），扩展"VIP 充值"等。绘制要点同上：登录=include；VIP 充值=extend。

2013-2014 期末 A · 综合四（医院取药告知系统 · 问题 4）
问题：针对病人使用查询机，确定该查询对应的操作契约及设计模型交互图（10 分）。
操作契约：

操作	queryMedicine(cardId)
交叉引用	查询取药状态
前置条件	病人已缴费并持有就医卡
后置条件	①查询机与处方建立关联（读取处方 ID）； ②查询机与窗口队列建立关联（通过处方 Id）； ③查询属性被赋值：病人姓名、窗口号、前面人数 / 是否超时

设计模型交互图属于第 7 章内容，需画 Controller → 业务对象 → DAO → DB 的分层调用。本章按 SSD 处理（只画 Actor + 系统）。

⑨ A4 双栏 Cheat Sheet (考前速查)

OOA 双模型

- **领域模型**: "有什么", UML 类图+活动图, 纯业务抽象
- **用例模型**: "做什么", 4 要素
- 用例模型 4 要素: **用例图 + 用例说明 + SSD + 操作契约** (前两者基础, 后两者附加)

UML 4+1 视图

- **用例视图** (核心, 用户视角): 用例图+活动图
- **逻辑视图** (静态结构): 类图/对象图/顺序图
- **进程视图** (并发同步): 状态图/活动图
- **构件视图** (代码组织): 构件图
- **部署视图** (硬件映射): 部署图

UML 9 种图速记

用类对顺协状活构部
(用例/类/对象/顺序/协作/状态/活动/构件/部署)

类六大关系 (弱→强)

- **依赖** 虚线箭头 (参数/局部使用)
- **关联** 实线 (属性持有, 可单向/双向)
- **聚合** 空心菱形 (部分可独立)
- **组合** 实心菱形 (生命周期依附)
- **泛化/继承** 空心三角实线
- **实现** 空心三角虚线

用例图三元素

- Actor (人/外部系统/设备/时钟)
- 用例 (椭圆, 动词+名词)
- 关联 (直线连接)
- 系统边界 (矩形)

include vs extend

- **include**: 必发, 基→子虚线箭头, 抽取公共步骤
- **extend**: 条件触发, 子→基虚线箭头, 抽取条件分支
- "不能改原用例"→用 extend
- 密码验证一般作 include (不作独立用例)

用例 3 大误区

1. 非目的性用例 (登录、密码验证不应独立)
2. 系统操作型用例 (DB 连接、数据通信)
3. 粒度过细 (增删改查应合并为"XX 管理")

SSD 三点

- 只画 Actor + 系统黑盒 (系统对象唯一)
- 消息: 名(参数)、返回《return》、loop、opt
- 不画 Controller/DAO/DB (那是设计顺序图)

操作契约后置条件三要素 (必背)

1. 对象创建或消除
2. 关联 (链) 建立或断开
3. 属性值修改

声明性, 描述状态变化, 不写算法。

SSD vs 设计顺序图

- SSD: 第 4 章, Actor+系统黑盒
- 设计顺序图: 第 7 章, 含 Controller/DAO/DB
- 关键词: GRASP/控制器/信息专家→设计顺序图
- 关键词: 系统事件/操作契约→SSD

概念类识别 4 法则

1. 名词短语法 (用例描述中划名词)
2. 能赋值→属性; 不能赋值→概念类
3. 举棋不定→作概念类
4. 排除软件类 (Controller/DAO/UI)

关联类型

- "需要知道"型 (须保存, 重点)
- "只需理解"型 (增强理解)
- 高优先级 3 种: A 是 B 一部分 / 包含依赖 / 描述

领域模型 vs 软件模型

- 领域模型 = 概念类 (业务世界名词)
- 软件模型 = 软件类 (解域, 含方法)
- 领域模型不是待构建软件模型的一部分

OOA vs OOD

- OOA: 领域模型 + 用例模型
- OOD: 设计模型 (体系结构+设计类图+交互图)
- 两者采用一致的 UML 表示法, **无缝衔接**
- "采用不同表示法"错

关键判断题速答

- 操作契约描述执行过程? × (描述后置状态变化)
- 业务流程用用例图? × (用活动图)
- 领域模型是软件模型一部分? ×
- 登录是独立用例? × (应作 include)
- 用例粒度越小越好? ×
- 控制器实现系统操作? × (仅接收)
- 顺序图能描述对象内部状态? × (用状态图)
- SSD 描述返回消息=后置条件? ×

Actor 不能是什么

不能是**数据** (被动信息, 不主动交互)。能是: 人、外部系统/设备、时钟 Timer。

领域模型构建 4 步

1. 找候选概念类
2. 描述概念类 (问题域命名)
3. 添加关联 (继承/聚合/组合)
4. 添加属性 (简单类型)

操作契约编写 4 步

1. SSD 识别系统事件
2. 结合领域模型找概念类
3. 对复杂/不清晰操作写契约
4. 按三要素写后置条件

综合大题 7 步流程

1. 定问题域边界
2. 找 Actor
3. 找用户目标级用例
4. 判 include/extend/泛化
5. 画用例图
6. 写用例说明 (成功+扩展)
7. 关键用例画 SSD + 操作契约

属性识别原则 - 简单类型： Boolean/date/number/string/text/time/address/color/zip - 属性值不应是对象 - 联系概念类用关联 - 可见性推迟到设计阶段
用例描述字段 编号/名称/范围/级别/参与者/相关人员/前置/后置/主要成功场景/扩展/*a/特殊需求/发生频率
必考填空速记 - 用例模型 4 要素：用例图、用例说明、SSD、操作契约 - 领域模型：UML 类图+活动图 - 后置条件三要素：创建/关联/属性 - SSD 两对象：Actor+系统黑盒 - UP 三特点：用例驱动、架构核心、迭代增量
聚合 vs 组合 一句话 聚合=部分可独立（学校-教师）；组合=部分随整体消亡（文档-章节）。
关联 vs 依赖 一句话 关联=属性长期持有（实线）；依赖=临时使用参数/局部（虚线）。

类图 3 区 类名（必需） / 属性 / 方法。领域模型只用前两种。 可见性 + public、- private、# protected、~ package
消息类型 - 同步：实心三角箭头实线 - 异步：开口箭头实线 - 返回：虚线 + 《return》 «create» / «destroy» 构造型
领域模型常用基数 1 实例 1 个 0..1 可选 1 个 * 任意多个 1..* 至少 1 个 - 明确写出来（评分点）
高频综合大题场景 - 医院挂号/就诊/取药（2007-2019 高频） - ATM 取款 - 电子商务 VIP 支付 - 网上订票订座 - 食堂一卡通 - 写诗机器人（2021-2022）